ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

UNIVERSIDAD

Software Architecture Laboratory

Laboratory No. 5
BluePrints API – JWT Security

OAuth 2.0 Resource Server / Spring Boot 3.3.x

Students:

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastian Ortega Muñoz

Instructor: Professor Javier Ivan Toquica Barrera

Course: Software Architecture (ARSW)

Institution: Escuela Colombiana de Ingeniería Julio Garavito

March 2026

0 Contents

1 Objective

This laboratory extends the Blueprints REST API developed in Laboratory 4 by incorporating a stateless security layer based on **JSON Web Tokens (JWT)** following the *OAuth 2.0 Resource Server* pattern.

Security Objectives

1. Configure Spring Security 6 as an OAuth2 Resource Server validating RS256-signed JWTs.
2. Implement an in-process token issuer (`/auth/login`) that generates short-lived tokens with fine-grained **scopes**.
3. Enforce **method-level access control** via `@PreAuthorize` distinguishing read (`blueprints.read`) from write (`blueprints.write`) operations.
4. Expose validated JWT authentication through the **Swagger UI** (OpenAPI 3 Bearer scheme).
5. Maintain the existing filter pipeline (Identity, Redundancy, Undersampling strategies) from the previous lab.
6. Achieve **full test coverage** with MockMvc JWT mocks and a real end-to-end authentication flow test.

2 Application Architecture

The application follows a layered architecture that adds a dedicated **Security Layer** on top of the REST layer introduced in Lab 4.

2.1 Package Structure

Package	Responsibility
`model`	Domain entities (`Blueprint`, `Point`)
`persistence`	Storage interface + in-memory impl
`filters`	Strategy pattern filter pipeline
`services`	Business orchestration
`api`	REST controllers + DTOs
`auth`	Token-issuing `AuthController`
`security`	Security config, JWT beans, user service
`config`	OpenAPI / Swagger configuration

2.2 Request / Response Flow

Every secured request traverses the following layers:

1. **SecurityFilterChain** — Spring Security intercepts the HTTP request. Public paths (`/auth/login`, Swagger) are permitted; all `/api/**` paths require a valid Bearer token.
2. **JwtDecoder** — validates the RSA-2048 signature and expiration of the incoming JWT.
3. **@PreAuthorize** — `MethodSecurityConfig` evaluates the scope claim; insufficient scope yields HTTP 403.
4. **Service layer** — business logic and filter application.
5. **Persistence layer** — `ConcurrentHashMap` storage.
6. **ApiResponse<T>** — uniform JSON envelope returned to the client.

Key Design Decision

The API does *not* rely on an external authorisation server. The same Spring Boot process generates the RSA key pair at startup, issues JWTs via `/auth/login`, and validates them via `JwtDecoder`. All key material is ephemeral (in-memory only).

3 Part 1 — Ported Domain Model

The complete domain and persistence layer from Lab 4 was ported into the security lab package structure (`co.edu.eci.blueprints`) without functional changes.

3.1 Domain Entities

```
public class Blueprint {
    private String author;
    private String name;
    private List<Point> points;
    // constructor, getters, toString
}

public class Point {
    private int x;
    private int y;
    // constructor, getters, toString, equals/
        hashCode
}
```

Listing 1: Blueprint and Point records

3.2 Persistence Interface

```
public interface BlueprintPersistence {
    void saveBlueprint(Blueprint bp)
        throws BlueprintPersistenceException;
    Blueprint getBlueprint(String author, String
        name)
        throws BlueprintNotFoundException;
    Set<Blueprint> getBlueprintsByAuthor(String
        author)
        throws BlueprintNotFoundException;
    Set<Blueprint> getAllBlueprints();
}
```

Listing 2: BlueprintPersistence interface

3.3 Seed Data

The `InMemoryBlueprintPersistence` bean pre-loads three blueprints:

Author	Name	Points
john	house	4
john	garage	3
jane	garden	3

3.4 Service Layer

`BlueprintsServices` orchestrates all business operations (get by author, get by name, list all, save) and delegates post-processing to the active `BlueprintsFilter` implementation.

4 Part 2 — JWT Security Implementation

4.1 RSA Key Generation

`JwtKeyProvider` generates a 2048-bit RSA key pair at application startup using the Java Security API:

```
@Component
public class JwtKeyProvider {
```

```
    private final RSAPublicKey  publicKey;
    private final RSAPrivateKey privateKey;

    public JwtKeyProvider() throws
         NoSuchAlgorithmException {
        KeyPairGenerator gen =
            KeyPairGenerator.getInstance("RSA");
        gen.initialize(2048);
        KeyPair pair = gen.generateKeyPair();
        this.publicKey  = (RSAPublicKey)  pair.
             getPublic();
        this.privateKey = (RSAPrivateKey) pair.
             getPrivate();
    }
}
```

Listing 3: JwtKeyProvider key generation

> **Note**
>
> The key pair is recreated on every restart. All previously issued tokens become immediately invalid after a restart, which is acceptable for a development / lab environment.

4.2 Security Configuration

`SecurityConfig` defines the Spring Security filter chain and the JWT infrastructure beans:

```
@Bean
public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {
    http
      .csrf(AbstractHttpConfigurer::disable)
      .sessionManagement(sm -> sm.
           sessionCreationPolicy(
           SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/auth/login",
              "/swagger-ui/**", "/v3/api-docs/**",
              "/actuator/**").permitAll()
          .anyRequest().authenticated())
      .oauth2ResourceServer(rs ->
          rs.jwt(Customizer.withDefaults()));
    return http.build();
}

@Bean
public JwtDecoder jwtDecoder(JwtKeyProvider keys)
    {
    return NimbusJwtDecoder
        .withPublicKey(keys.getPublicKey()).build
            ();
}

@Bean
public JwtEncoder jwtEncoder(JwtKeyProvider keys)
    {
    JWK jwk = new RSAKey.Builder(keys.getPublicKey
         ())
        .privateKey(keys.getPrivateKey()).build();
    return new NimbusJwtEncoder(
        new ImmutableJWKSet<>(new JWKSet(jwk)));
}
```

Listing 4: SecurityConfig (simplified)

4.3 User Service

`InMemoryUserService` implements `UserDetailsService` with two hard-coded users (suitable for a lab environment):

Username	Password	Role
admin	*bcrypt hash*	USER
user	*bcrypt hash*	USER

4.4 Token Issuer — AuthController

`POST /auth/login` validates credentials and returns a signed JWT:

```
@PostMapping("/auth/login")
public ResponseEntity<TokenResponse> login(
        @RequestBody LoginRequest req) {
    // 1. Load user and verify BCrypt password
    // 2. Build JWT claims
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("blueprints-api")
        .subject(user.getUsername())
        .issuedAt(now)
        .expiresAt(now.plusSeconds(3600))
        .claim("scope",
            "blueprints.read blueprints.write")
        .build();
    // 3. Sign and return
    return ResponseEntity.ok(new TokenResponse(
        jwtEncoder.encode(...).getTokenValue(),
        "Bearer", 3600));
}
```

Listing 5: AuthController token issuance

4.5 JWT Scopes

Scope	Allowed Operations
`blueprints.read`	All GET endpoints
`blueprints.write`	POST and PUT endpoints

4.6 Evidence: Application Startup

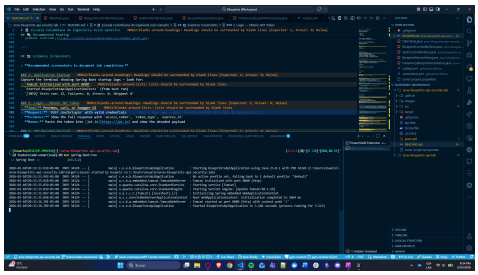

Figure 1: Spring Boot startup — RSA key pair generated, context ready

4.7 Evidence: Login and JWT

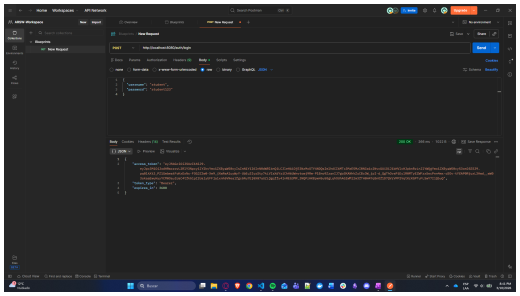

Figure 2: `POST /auth/login` — 200 OK with JWT access token

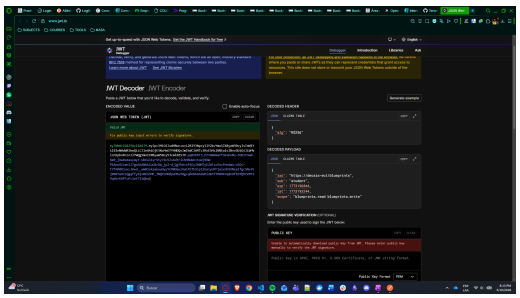

Figure 3: JWT payload decoded at `jwt.io` — subject, scope, expiry

5 Part 3 — Protected REST API

5.1 Endpoint Overview

All endpoints are defined in `BlueprintController` under the `/api/blueprints` base path and protected with `@PreAuthorize`:

Path	Method	Required Scope
`/api/blueprints`	GET	`blueprints.read`
`/api/blueprints/{author}`	GET	`blueprints.read`
`/api/blueprints/{a}/{b}`	GET	`blueprints.read`
`/api/blueprints`	POST	`blueprints.write`
`/api/blueprints/{a}/{b}`	PUT	`blueprints.write`

5.2 Method-Level Security

```
@PreAuthorize(
  "hasAuthority('SCOPE_blueprints.read')")
@GetMapping
public ResponseEntity<ApiResponse<
    Set<Blueprint>>> getAllBlueprints() { ... }

@PreAuthorize(
  "hasAuthority('SCOPE_blueprints.write')")
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public ResponseEntity<ApiResponse<Blueprint>>
    createBlueprint(
        @Valid @RequestBody
        NewBlueprintRequest req) { ... }
```

Listing 6: @PreAuthorize on GET and POST

5.3 Uniform Response Envelope

All responses are wrapped in `ApiResponse<T>`:

```
public record ApiResponse<T>(
    int    code,
    String message,
    T      data
) {}
```

Listing 7: ApiResponse record

5.4 HTTP Status Semantics

Status	Condition
200 OK	Successful read
201 Created	Blueprint saved
400	Validation failure
401	Missing / invalid JWT
403	Insufficient scope
404	Blueprint not found
500	Unexpected server error

5.5 Evidence: Authorised GET

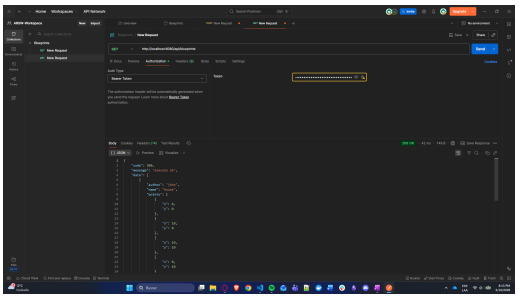

Figure 4: `GET /api/blueprints` with valid Bearer token — 200 OK

5.6 Evidence: Unauthorised GET (no token)

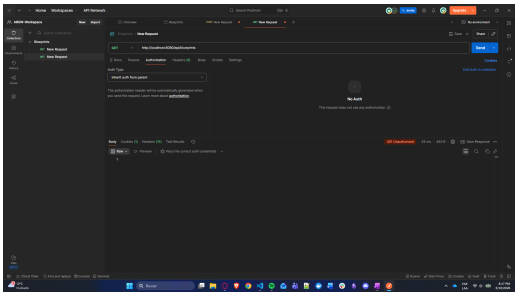

Figure 5: `GET /api/blueprints` without token — 401 Unauthorized

5.7 Evidence: Authorised POST

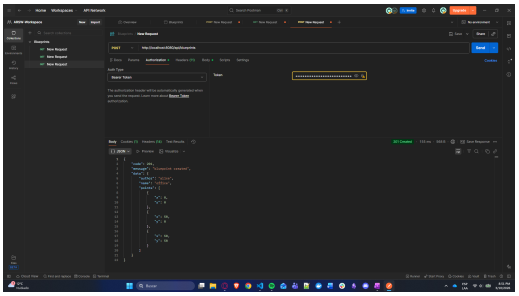

Figure 6: `POST /api/blueprints` with write scope — 201 Created

5.8 Evidence: Forbidden POST (read-only scope)

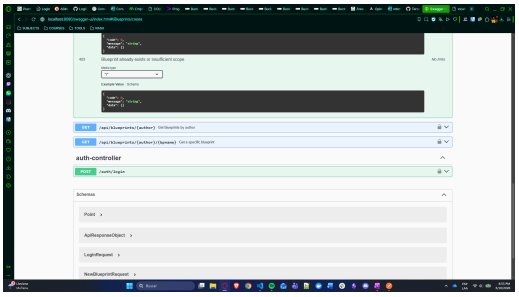

Figure 7: `POST /api/blueprints` with read-only scope — 403 Forbidden

6 Part 4 — OpenAPI / Swagger with JWT

6.1 OpenAPI Security Scheme

`OpenApiConfig` registers a *Bearer* security scheme so that Swagger UI can attach the JWT to every request:

```
@Bean
public OpenAPI customOpenAPI() {
    return new OpenAPI()
      .info(new Info()
          .title("Blueprints API Security")
          .version("2.0.0")
          .description(
              "JWT-secured REST API (RS256)"))
      .addSecurityItem(new SecurityRequirement()
          .addList("bearerAuth"))
      .components(new Components()
          .addSecuritySchemes("bearerAuth",
              new SecurityScheme()
                  .type(SecurityScheme.Type.HTTP)
                  .scheme("bearer")
                  .bearerFormat("JWT")));
}
```

Listing 8: OpenApiConfig Bearer scheme

6.2 Endpoint Annotations

Every controller method is annotated with:

- `@Operation(summary=...)` — human-readable description.
- `@ApiResponses` — documents 200/201, 401, 403, 404, 500.
- `@SecurityRequirement(name="bearerAuth")` — tells Swagger to add the `Authorization: Bearer` header.
- `@Tag(name="Blueprints")` — groups endpoints in the UI.

6.3 Evidence: Swagger Authorize Dialog

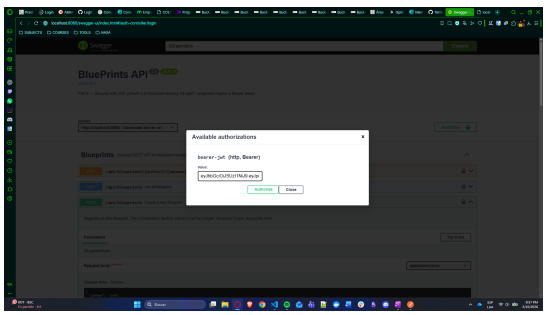

Figure 8: Swagger UI Authorize dialog for Bearer JWT entry

6.4 Evidence: Executing a Secured Endpoint

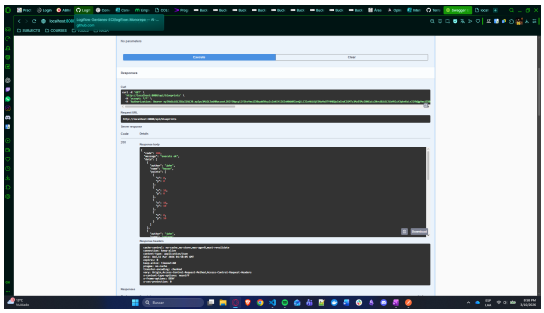

Figure 9: Swagger UI executing a secured endpoint with the authorisation token

7 Part 5 — Filter Pipeline

The filter pipeline from Lab 4 was retained without modification. It implements the **Strategy design pattern** to post-process blueprint point lists:

```
public interface BlueprintsFilter {
    Blueprint filter(Blueprint bp);
}
```

Listing 9: BlueprintsFilter interface

7.1 Available Strategies

Filter	Behaviour
`IdentityFilter`	Returns the original point list unchanged (default profile).
`RedundancyFilter`	Removes consecutive duplicate points (`@Profile("redundancy")`).
`UndersamplingFilter`	Keeps every other point to reduce resolution (`@Profile("undersampling")`).

7.2 Profile Activation

The active filter is selected via Spring profiles in `application.yml`:

```
spring:
  profiles:
    active: default    # use 'redundancy' or
                       # 'undersampling' to switch
```

Listing 10: Profile activation in application.yml

Note

`IdentityFilter` uses `@Profile("{!redundancy & !undersampling}")` so it engages automatically whenever neither alternative profile is active.

8 Testing Strategy

A total of **32 tests** across 5 test classes verify functional correctness and security enforcement:

Test Class	Tests
`BlueprintServiceTests`	7
`AuthControllerTests`	6
`BlueprintControllerTests`	14
`FilterTests`	5
Total	**32**

8.1 JWT Mock Technique

`BlueprintControllerTests` uses `SecurityMockMvcRequestPostProcessors.jwt()` to inject synthetic JWT-backed `Authentication` objects without ever calling `/auth/login`:

```
// Read scope only  -> 200
mockMvc.perform(get("/api/blueprints")
    .with(jwt().authorities(
        new SimpleGrantedAuthority(
            "SCOPE_blueprints.read"))))
    .andExpect(status().isOk());

// Write scope -> 201
mockMvc.perform(post("/api/blueprints")
    .with(jwt().authorities(
        new SimpleGrantedAuthority(
            "SCOPE_blueprints.write")))
    .contentType(APPLICATION_JSON)
    .content(payload))
    .andExpect(status().isCreated());

// Read scope on POST -> 403
mockMvc.perform(post("/api/blueprints")
    .with(jwt().authorities(
        new SimpleGrantedAuthority(
            "SCOPE_blueprints.read")))
    .contentType(APPLICATION_JSON)
    .content(payload))
    .andExpect(status().isForbidden());
```

Listing 11: JWT mock for scope testing

8.2 End-to-End Authentication Flow

One dedicated test exercises the full token lifecycle: login, token extraction, and a secured request using the *real* token from the real `JwtEncoder`:

```
@Test
void fullAuthFlow_loginThenAccessApi() throws
     Exception {
    // Step 1: obtain real JWT
    MvcResult loginResult =
        mockMvc.perform(post("/auth/login")
            .contentType(APPLICATION_JSON)
            .content("{\"username\":\"admin\"," +
                     "\"password\":\"admin123\"}")
                      )
        .andExpect(status().isOk())
        .andReturn();

    String token = /* extract from response body
         */;

    // Step 2: use real JWT on secured endpoint
    mockMvc.perform(get("/api/blueprints")
        .header("Authorization", "Bearer " + token
            ))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value(200));
}
```

Listing 12: Real login -¿ real token -¿ real request test

8.3 Evidence: All Tests Passing

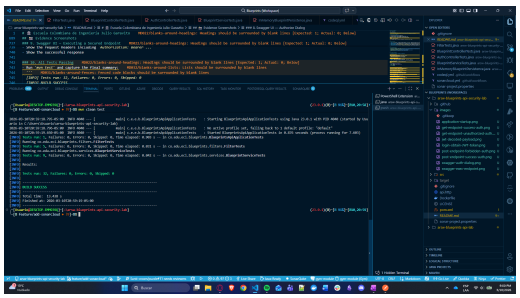

Figure 10: Maven Surefire output — Tests run: 32, Failures: 0, BUILD SUCCESS

Test Run Summary

- 32 tests executed across 5 test classes.
- 0 failures, 0 errors, 0 skipped.
- Scope enforcement (read/write/none) fully verified.
- End-to-end real JWT flow verified.
- Filter strategies independently tested.
- Maven build: **BUILD SUCCESS**.

9 Conclusions

Key Findings

1. **Stateless security** via JWT eliminates server-side session state, making the API horizontally scalable with no shared session store.
2. **RS256 signing** with an ephemeral RSA key pair ensures that tokens cannot be forged, even without a dedicated key-management service.
3. **Scope-based access control** is more expressive than simple role-based control: the same user account can hold both `blueprints.read` and `blueprints.write`, or only the read scope, without requiring multiple user records.
4. `@PreAuthorize` at the method level is the correct Spring Security 6 approach, keeping security declarations co-located with business logic rather than scattered across a central configuration.
5. **MockMvc JWT mocks** (`SecurityMockMvcRequestPostProcessors.jwt()`) allow fast, deterministic unit testing of access-control rules without spinning up an authorisation server.
6. **Separation of concerns** is preserved: the security layer adds zero coupling to the domain, persistence, or filter layers, as evidenced by unmodified service and persistence tests.

9.1 Lessons Learned

- `GlobalExceptionHandler` must rethrow `AccessDeniedException` and `AuthenticationException` so that Spring Security's `ExceptionTranslationFilter` can convert

them to 403/401 responses. Catching them inside `@RestControllerAdvice` produces incorrect 500 responses.

- Removing the external `jwk-set-uri` and relying on the locally configured `JwtDecoder` bean avoids startup failures caused by unavailable remote JWKS endpoints during development.
- Spring Security auto-configures an `oauth2ResourceServer` only when a `JwtDecoder` bean is present; explicitly declaring `.oauth2ResourceServer(rs -> rs.jwt(...))` in the filter chain definition avoids ambiguity with the auto-configuration.

10 References

1. Spring Security Reference — OAuth2 Resource Server. https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
2. RFC 7519 — JSON Web Token (JWT). IETF. https://datatracker.ietf.org/doc/html/rfc7519
3. Nimbus JOSE + JWT library. https://connect2id.com/products/nimbus-jose-jwt
4. Spring Boot 3.3 Release Notes. https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.3-Release-Notes
5. SpringDoc OpenAPI — `springdoc-openapi-starter-webmvc-ui`. https://springdoc.org
6. JWT.io — JWT decoder and debugger. https://jwt.io
7. Baeldung — Spring Security Test with JWT. https://www.baeldung.com/spring-security-test-oauth2